You don't have to give the block parameter a name if you are only going to pass it along, you can just use a bare & character.

```
class Child < Parent
  def do_something(&)
    do_something_else(&)
  end
end
```

Combining all these mechanisms, if you just want to roll all the arguments of a method along to a different method, then def(*args, **kwargs, &block) is an awkward way to gather all the arguments. Ruby has a simpler way.

```
class Thing
  def do_something(...)
    do_something_else(...)
  end
end
```

The triple dot syntax is an anonymous way to pass all arguments to one method onward to a different method.

Calling a Method

You call a method by optionally specifying a receiver, giving the name of the method, and optionally passing some arguments and an optional block. Here's a code fragment that shows us calling a method with a receiver, a positional argument, a keyword argument and a block:

```
connection.download_mp3("jitterbug", speed: :slow) { |p| show_progress(p) }
```

In this example, the object connection is the receiver, download_mp3 is the name of the method, the string "jitterbug" is the positional parameter, the key/value pair speed: :slow is a keyword parameter, and the code between the braces is the associated block argument. When the method is called, Ruby invokes the method in that object, and inside that method, self is set to that receiver object. For class and module methods, the receiver will be the class or module object.

```
File.size("testfile") # => 66
Math.sin(Math::PI/4)  # => 0.7071067811865475
```

Ruby allows you to omit the receiver, in which case the default receiver is self, the current object. In this example, all the methods in write_on are called with the current object as the implicit receiver.

```
class InvoiceWriter
  def initialize(order)
    @order = order
  end

  def write_on(output)
    # called on current object, as there is no reciever
    write_header_on(output)
    write_body_on(output)
    write_totals_on(output)
  end
```